Часть 2. Массивы, строки, методы • Занятие 2 из 2

2.4. Методы

К концу занятия ты сможешь:

- вызывать один метод с разным набором значений;
- передавать в метод сколько угодно чисел;
- раскладывать программу по методам: один метод — одна задача.

Разминка • 2.4, занятие 1

Вспомни прошлое занятие

1. Что выведет?

```
Console.WriteLine(Twice(4));
static int Twice(int number)
{
    return number * 2;
}
```

2. В чём разница?

Чем метод **static void Greet()** отличается от **static int Bonus()**?

Что можно сделать с результатом каждого из них?

3. Найди ошибку

```
Console.WriteLine(clicks);
static void Count()
{
    int clicks = 5;
}
```

Задача на сегодня

Как написать это всего двумя методами?

Program.cs

```
PrintLine();
PrintLine(30);
PrintLine(30, '*');
Console.WriteLine(Sum(2, 3));
Console.WriteLine(Sum(1, 2, 3, 4, 5));
```

Вывод в консоли

```
------------------
------------------------------
******************************
5
15
```

Сейчас ты бы написал пять разных методов: для линии из 20 символов, из 30, из звёздочек, для суммы двух чисел, пяти чисел…

К концу занятия хватит двух: PrintLine и Sum.

План занятия

Два урока по 40 минут

Урок 1

- Разминка и план — 5 мин
- Перегрузка: одно имя, разные версии
- Необязательные и именованные параметры
- Вопросы
- Практика «Калькулятор ремонта»

Урок 2

- params: сколько угодно значений
- Один метод — одна задача
- Вопросы и практика «Статистика матча»
- Мини-проект «Анализатор плейлиста»
- Итоги темы 2.4

Вспомним • 2.4, занятие 1

Из чего состоит метод

Program.cs

```
Console.WriteLine(Twice(4));  // 8

static int Twice(int number)
{
    return number * 2;
}
```

static — пока просто запомни (подробно — в 4.3)

int — тип результата; **void** — без результата

Twice — имя метода

int number — параметр: значение, которое передают в метод

return — вернуть результат наружу

Свои методы пишем ниже основного кода. Переменная, объявленная внутри метода, видна только внутри него.

Блок 1 из 2 • Урок 1

Один метод — разные способы вызова

В этом блоке ты научишься:

- узнавать разные версии одного метода и писать свои;
- задавать параметрам значения по умолчанию;
- подписывать значения при вызове именем параметра.

Перегрузка: одно имя — несколько версий

Перегрузка — несколько версий метода с одним именем, но с разными параметрами. C# сам выбирает подходящую.

Как кнопка «Отправить» в мессенджере: одна кнопка, а отправляет и текст, и фото, и голосовое.

Без перегрузки пришлось бы помнить WriteLineInt, WriteLineString, WriteLineDouble…

Program.cs

```
Console.WriteLine(5);      // версия для int
Console.WriteLine("5");    // версия для string
Console.WriteLine(2.5);    // версия для double
```

Вывод в консоли

```
5
5
2,5
```

Место для скриншота: подсказка Visual Studio после ввода Console.WriteLine(— «1 из N» и стрелки вверх-вниз

Как C# выбирает версию

По количеству и типам значений в скобках. У Math.Max есть версия для int и версия для double — и они возвращают разные типы.

Program.cs

```
Console.WriteLine(Math.Max(7, 2) / 2);
Console.WriteLine(Math.Max(7.0, 2) / 2);
```

Вывод в консоли

```
3
3,5
```

Вызов	Какая версия	Что дальше
Math.Max(7, 2)	для int: оба числа целые	7 / 2 — целое деление: 3
Math.Max(7.0, 2)	для double: 2 станет 2.0	7.0 / 2 — дробное: 3,5

Правило: версию выбирают типы значений, а не то, что ты хотел получить.

Ловушка: две версии своего Area — ошибка

Program.cs

```
Console.WriteLine(Area(3));
Console.WriteLine(Area(4, 5));
static double Area(double radius)
{
    return Math.PI * radius * radius;
}
static double Area(double width, double height)
{
    return width * height;
}
```

Ошибка при сборке

```
error CS0128: A local variable or function named 'Area'
is already defined in this scope
```

Перевод: локальная переменная или функция с именем 'Area' уже объявлена в этой области.

Почему: методы ниже основного кода C# считает **локальными функциями** — методами, которые живут внутри основного кода. Локальные функции перегружать нельзя.

Console.WriteLine перегружать можно, а свой метод в обычном Program.cs — нет. Как быть — на следующем слайде.

Своя перегрузка: блок partial class Program

Program.cs

```
Console.WriteLine($"Ковёр: {Area(1.5):F2} м²");
Console.WriteLine($"Комната: {Area(4, 3.5):F2} м²");

partial class Program   // пока просто запомни
{
    static double Area(double radius)
    {
        return Math.PI * radius * radius;
    }
    static double Area(double width, double height)
    {
        return width * height;
    }
}
```

Перегруженные методы кладём в блок **partial class Program { … }** в самом конце файла.

Внутри — обычные static-методы. Основной код вызывает их как раньше.

Что такое class и partial — подробно в теме 4.1.

Результат: C# сам выбрал версию

Вывод в консоли

```
Ковёр: 7,07 м²
Комната: 14,00 м²
```

Вызов	Какая версия сработала
Area(1.5)	Area(double radius) — одно значение
Area(4, 3.5)	Area(double width, double height) — два значения, 4 станет 4.0

Границы: версии должны отличаться **количеством** или **типами** параметров. Разные имена параметров не в счёт: Area(double baseLength, double height) и Area(double width, double height) для C# одинаковы — снова ошибка.

Правило: одно имя — разные наборы параметров.

Необязательный параметр: значение по умолчанию

Необязательный параметр — параметр, у которого есть значение по умолчанию. Не передал значение — берётся оно. Как в кофейне: не назвал размер — дадут средний.

Запись: **тип имя = значение**. Без этого пришлось бы писать три метода или всегда передавать все значения.

Program.cs

```
PrintLine();          // оба по умолчанию
PrintLine(10);        // symbol по умолчанию
PrintLine(10, '*');   // оба свои
static void PrintLine(int length = 20, char symbol = '-')
{
    for (int i = 0; i < length; i++)
        Console.Write(symbol);
    Console.WriteLine();
}
```

Вывод в консоли

```
--------------------
----------
**********
```

Необязательные параметры — только в конце

Ошибка • Program.cs

```
PrintLine(10);
static void PrintLine(char symbol = '-', int length)
{
    for (int i = 0; i < length; i++)
        Console.Write(symbol);
    Console.WriteLine();
}
```

Ошибка при сборке

```
error CS1737: Optional parameters must appear after all required parameters
```

Перевод: необязательные параметры должны идти после всех обязательных.

Правильно • Program.cs

```
PrintLine(10);
static void PrintLine(int length, char symbol = '-')
{
    for (int i = 0; i < length; i++)
        Console.Write(symbol);
    Console.WriteLine();
}
```

Вывод в консоли

```
----------
```

Правило: сначала обязательные параметры, потом необязательные.

Именованные аргументы: подписываем значения

Именованный аргумент — значение с подписью: **имя параметра: значение**. Так можно пропустить параметр в середине и не бояться перепутать порядок.

Program.cs

```
PrintLine(symbol: '=');
PrintLine(length: 5, symbol: '#');
PrintLine(symbol: '~', length: 8);
static void PrintLine(int length = 20, char symbol = '-')
{
    for (int i = 0; i < length; i++)
        Console.Write(symbol);
    Console.WriteLine();
}
```

Вывод в консоли

```
====================
#####
~~~~~~~~
```

Имя должно совпадать с параметром: PrintLine(size: 5) — ошибка CS1739.

Правило: подписывай значение, если без подписи непонятно, что оно значит.

Блок 1 • Сравнение

Перегрузка или необязательный параметр?

	Перегрузка	Необязательный параметр
Когда	данные разные: радиус или две стороны	задача та же, есть обычная настройка
Пример	Area(radius) и Area(width, height)	PrintLine(length = 20, symbol = '-')
Сколько версий	несколько методов с одним именем	один метод
Где писать сейчас	в partial class Program	ниже основного кода, как обычно

Правило: разные данные — перегрузка; та же задача с настройкой — необязательный параметр.

Живой кодинг: заголовок в рамке

1. Пишем метод PrintTitle(string text): рамка из «=» вокруг текста.
2. Добавляем необязательный параметр char border = '='.
3. Вызываем со своим символом и с именованными аргументами в другом порядке.
4. Специально ошибаемся: PrintTitle(42). Читаем сообщение и исправляем.

Что получится • Вывод в консоли

```
====================
= Таблица рекордов =
====================
```

Место для скриншота: окно «Список ошибок» с ошибкой CS1503

Блок 1 • Вопросы

Проверь себя

1. Что выведет?

```
Console.WriteLine(
    Math.Max(5, 2) / 2);
Console.WriteLine(
    Math.Max(5.0, 2) / 2);
```

2. Почему?

Почему две версии своего Area в обычном Program.cs дают ошибку CS0128? Как всё-таки сделать две версии?

3. Что выведет?

```
PrintLine(3);
static void PrintLine(
    int length = 20,
    char symbol = '-')
// тело как на слайде
```

Проверь себя: ещё два вопроса

4. Найди ошибку

```
Console.WriteLine(Bonus(coins: 50));
static int Bonus(int level = 1, int coins)
{
    return level * coins;
}
```

5. Что будет, если…

```
static double Cost(double area,
    double pricePerMeter = 500,
    int discountPercent = 0)
```

…вызвать Cost(54, 10) вместо Cost(54, discountPercent: 10)?

Задание • База

Калькулятор ремонта

- Спроси длину и ширину комнаты в метрах.
- Метод RoomArea(double length, double width) возвращает площадь.
- Метод Cost(double area, double pricePerMeter = 500, int discountPercent = 0) возвращает цену ламината.
- Выведи площадь и три цены: обычную, за ламинат по 800 руб., обычную со скидкой 10% (скидку передай по имени).

Подсказка: цена = площадь × цена за метр × (100 – скидка) / 100. Цены выводи через {…:F0}, площадь — через {…:F2}. Дробные вводи через запятую.

★ **Успел раньше?** Добавь параметр bool withDelivery = false: с доставкой цена больше на 1500 руб.

Застрял — 15 минут ищешь сам, потом спрашиваешь: что делал, что ожидал, что получил.

Задание • База

Что должно получиться

Вывод в консоли

```
Длина комнаты в метрах: 4,5   <- ввёл пользователь
Ширина комнаты в метрах: 3   <- ввёл пользователь
Площадь: 13,50 м²
Обычный ламинат: 6750 руб.
Ламинат за 800 руб.: 10800 руб.
Со скидкой 10%: 6075 руб.
```

Стрелка — пометка для тебя, в программе её нет.

Проверь себя

- Комната 4 на 5: площадь 20,00, обычная цена 10000 руб.
- Скидка передана по имени, а не третьим числом подряд.
- Ввёл 4.5 с точкой и получил ошибку? Вводи через запятую (тема 1.2).

★★ Задача на смекалку

Семейство Max

Через partial class Program напиши три версии своего метода Max:

- для двух int;
- для трёх int;
- для массива int[].

Условие: версия для трёх чисел не сравнивает сама — она только вызывает версию для двух. Сколько раз ей это понадобится?

Вывод в консоли

```
Max(3, 9) = 9
Max(4, 12, 7) = 12
Max(массив 5, 1, 8) = 8
```

Блок 2 из 2 • Урок 2

params и порядок в коде

В этом блоке ты научишься:

- передавать в метод сколько угодно значений;
- не падать, когда значений нет совсем;
- раскладывать длинную программу по методам.

Блок 2 • params

params: сколько угодно значений

params перед массивом-параметром разрешает передавать значения просто через запятую — сколько угодно, даже ни одного. C# сам соберёт их в массив. Как пакет в магазине: кладёшь столько покупок, сколько купил.

Без params — Sum для двух, для трёх, для четырёх чисел… или каждый раз писать Sum(new int[] { 1, 2, 3 }).

Запись

```
static int Sum(params int[] numbers)
```

params — слово-разрешение • int[] — массив целых • numbers — имя массива внутри метода

Границы: params — только у последнего параметра и только один на метод. Обычный массив тоже можно передать: Sum(scores).

Пример: одна Sum на любое количество чисел

Program.cs

```
Console.WriteLine(Sum(1, 2));
Console.WriteLine(Sum(4, 8, 15, 16));
Console.WriteLine(Sum());    // ни одного числа

static int Sum(params int[] numbers)
{
    int total = 0;
    foreach (int number in numbers)
        total += number;
    return total;
}
```

Вывод в консоли

```
3
43
0
```

Внутри метода numbers — обычный массив (тема 2.1).

Без значений массив пустой: Length = 0, цикл не выполнится ни разу, вернётся 0.

Блок 2 • Ловушка

Ловушка: среднее, когда чисел нет

Program.cs

```
Console.WriteLine(Average(5, 3, 4));
Console.WriteLine(Average());

static int Average(params int[] numbers)
{
    int total = 0;
    foreach (int number in numbers)
        total += number;
    return total / numbers.Length;
}
```

Вывод в консоли

```
4

Unhandled exception. System.DivideByZeroException: Attempted to
divide by zero.
```

Перевод: необработанное исключение — попытка деления на ноль.

Дальше в консоли — строки с местом ошибки.

Почему: Average() — пустой массив, numbers.Length = 0, а делить на 0 нельзя. Как правильно — в самом начале метода: **if (numbers.Length == 0) return 0;**

Правило: метод с params должен работать и без значений.

Живой кодинг: один метод — одна задача

1. Смотрим «Викторину», где всё в основном коде: вопросы, проверка, итог.
2. Выносим AskQuestion(question, answer) — задаёт вопрос, возвращает 1 или 0.
3. Выносим CheckAnswer(userAnswer, answer) — только сравнивает.
4. Выносим PrintResult(score, total) — только выводит итог.
5. Специально ошибаемся: в PrintResult берём userAnswer из другого метода. Читаем ошибку, исправляем.

Стало • основной код

```
int score = 0;
score += AskQuestion("Сколько будет 7 * 8?", "56");
score += AskQuestion("Столица Франции?", "Париж");
PrintResult(score, 2);
```

Вывод программы до и после одинаковый. Меняется другое: основной код читается как план, а ошибку легко найти в маленьком методе.

Блок 2 • Один метод — одна задача

Как понять, что метод делает слишком много

«И» в названии

ReadAndCheckAndPrint — это три задачи, значит, три метода: Read, Check, Print.

Не влезает в экран

Чтобы прочитать метод, приходится прокручивать — его трудно понять целиком.

Комментарии-разделители

// ввод, // подсчёт, // вывод внутри одного метода — готовые имена для новых методов.

Правило: имя метода — глагол и одна задача: ParseTime, PrintTrack, CheckAnswer.

Блок 2 • Вопросы

Проверь себя

1. Что вернёт?

Что вернёт Sum() без чисел? Почему программа не падает, а Average() падает?

2. Найди ошибку

```
static void Report(
    params int[] scores,
    string player)
```

Почему так нельзя?

3. Что не так?

Что не так с методом ReadAndCheckAndPrint? На какие методы его разделить?

Задание • База

Статистика матча

- Метод BestScore(params int[] scores) возвращает лучший результат, а без чисел — 0.
- Метод Average(params int[] scores) возвращает среднее типа double, а без чисел — 0.
- Выведи статистику для очков 12, 25, 8, 19 и для матча без очков.

Подсказка: чтобы среднее было дробным, приведи сумму к double: (double)total / scores.Length (тема 1.2). Среднее выводи через {…:F1}.

★ **Успел раньше?** Метод PrintReport(string player, params int[] scores) печатает строку: «Даша: лучший 25, среднее 16,0».

Задание • База

Что должно получиться

Вывод в консоли • База

```
Лучший результат: 25
Средний результат: 16,0
Матч без очков: лучший 0, среднее 0,0
```

Вывод в консоли • ★

```
Даша: лучший 25, среднее 16,0
```

Проверь себя

- Вызов без чисел не роняет программу.
- Average(12, 13) даёт 12,5, а не 12.
- BestScore(-5, -2) даёт -2, а не 0.

★★ Задача на смекалку

Свой Format

Напиши метод Fill(string template, params string[] values). Он заменяет в шаблоне {0} на первое значение, {1} — на второе и так далее. Значений может быть сколько угодно. Пригодятся Replace (тема 2.3) и цикл.

Program.cs • вызовы

```
Console.WriteLine(Fill("Игрок {0} набрал {1} очков", "Shadow_Fox", "120"));
Console.WriteLine(Fill("{0} против {1}: {2}", "Лисы", "Совы", "3:2"));
```

Вывод в консоли

```
Игрок Shadow_Fox набрал 120 очков
Лисы против Совы: 3:2
```

Финальные вопросы • Вся тема 2.4

Свяжи всё вместе

1. params и массивы

Почему внутри Sum(params int[] numbers) работают foreach и numbers.Length? С какой темой это связано?

2. Строки в методе

Что вернёт "3:45".Split(':')? Зачем после этого нужен int.Parse?

3. Что будет, если поменять

В PrintLine(int length = 20, char symbol = '-') убрали «= 20». Какие вызовы сломаются?

4. Перегрузка

Почему для своей перегрузки понадобился partial class Program, а для необязательных параметров — нет?

Мини-проект • Идея

Анализатор плейлиста

Вводишь треки и их длительность — программа показывает список, общее время и среднюю длину трека.

Треки длиннее 4 минут отмечены звёздочкой.

Стрелка — пометка для тебя, в программе её нет.

Вывод в консоли

```
Сколько треков в плейлисте? 3  <- ввёл
Название трека 1: Night Drive  <- ввёл
Длительность (м:сс): 3:45  <- ввёл
Название трека 2: Summer  <- ввёл
Длительность (м:сс): 4:05  <- ввёл
Название трека 3: Intro  <- ввёл
Длительность (м:сс): 0:58  <- ввёл
> Night Drive — 3:45
* Summer — 4:05
> Intro — 0:58
Всего: 8:48
Средний трек: 2:56
```

Мини-проект • Что умеет

Что должна уметь программа

Принимать треки

Любое число треков, длительность в виде 3:45.

Показывать список

Каждый трек со временем; длиннее 4 минут — со звёздочкой.

Считать время

Общее время плейлиста и средняя длина трека.

Не падать

Если треков 0 — пишет «Плейлист пуст».

Это пригодится в Проекте 2: игра на поле тоже делится на методы — отрисовка, ход игрока, проверка победы.

Мини-проект • План

План по шагам

Шаг	Что сделать	Что нужно	Как проверить
1	ParseTime: "3:45" → 225 секунд	Split — 2.3; int.Parse — 1.2	ParseTime("3:45") выводит 225
2	FormatTime: 245 → "4:05"	/ и % — 1.3; if — 1.4	245 даёт 4:05, а не 4:5
3	TotalSeconds: сумма всех длительностей	params — сегодня; foreach — 2.1	TotalSeconds(225, 245, 58) выводит 528
4	PrintTrack и вывод: длинные — с marker: '*'; всего и среднее; пустой плейлист	необязательный и именованный — сегодня; if — 1.4	Пример со слайда; 0 треков — «Плейлист пуст»

Каждый шаг проверяй запуском сразу, не жди конца. Застрял — 15 минут ищешь сам, потом спрашиваешь.

Мини-проект • Заготовка 1 из 2

Основной код

Program.cs • начало файла

```
Console.Write("Сколько треков в плейлисте? ");
int count = int.Parse(Console.ReadLine());
string[] titles = new string[count];
int[] durations = new int[count];
for (int i = 0; i < count; i++)
{
    Console.Write("Название трека " + (i + 1) + ": ");
    titles[i] = Console.ReadLine();
    Console.Write("Длительность (м:сс): ");
    durations[i] = ParseTime(Console.ReadLine());
}
// Шаг 4: выведи каждый трек через PrintTrack; длиннее 4:00 — с marker: '*'
// Шаг 4: если треков 0 — "Плейлист пуст", иначе выведи всего и среднее
```

Мини-проект • Заготовка 2 из 2

Методы: допиши шаги 1–4

Program.cs • ниже основного кода

```
static int ParseTime(string text)
{
    string[] parts = text.Split(':');   // "3:45" -> "3" и "45"
    // Шаг 1: верни минуты * 60 + секунды
}
static string FormatTime(int seconds)
{   // Шаг 2: 225 -> "3:45", 245 -> "4:05" (а не "4:5")
}
static int TotalSeconds(params int[] durations)
{   // Шаг 3: сложи все длительности
}
static void PrintTrack(string title, int seconds, char marker = '>')
{   // Шаг 4: печатает "> Night Drive — 3:45"
}
```

Пока шаг не дописан, Visual Studio подчёркивает метод красным: ему нечего вернуть. Это нормально — допиши return.

Мини-проект • Проверка

Готово, если…

- с треками из примера выводится «Всего: 8:48» и «Средний трек: 2:56»;
- 4:05 печатается как 4:05, а не 4:5;
- Summer (4:05) со звёздочкой, Intro — без;
- если ввести 0 треков, программа пишет «Плейлист пуст» и не падает.

★ Неверный ввод

Длительность «abc» или «345» без двоеточия не роняет программу: «Не понял длительность, считаю 0:00». Пригодится TryParse (1.2).

★★ Самый длинный и остаток

Выведи «Самый длинный: Summer (4:05)». Спроси номер трека и покажи, сколько играть с него до конца: «С трека 2 до конца: 5:03».

Объясни каждую строку своего кода. Сохрани работу в Git и выложи на GitHub.

Челлендж

Рамка для текста

Метод PrintBox(params string[] lines) рисует рамку вокруг любого числа строк. Ширина рамки — по самой длинной строке, короткие строки дополняются пробелами.

Потом добавь вторую версию PrintBox(char border, params string[] lines) — со своим символом рамки. Где её писать, чтобы не было ошибки CS0128?

Program.cs • вызов

```
PrintBox("Счёт", "Лисы 3 : 2 Совы", "Финал");
```

Вывод в консоли

```
+-----------------+
| Счёт            |
| Лисы 3 : 2 Совы |
| Финал           |
+-----------------+
```

Итоги темы 2.4

Теперь я умею...

- разбивать программу на методы: один метод — одна задача;
- передавать в метод данные через параметры и получать результат через return;
- понимать, где переменная видна, а где нет;
- выбирать нужную версию перегруженного метода и писать свою;
- задавать параметрам значения по умолчанию и подписывать значения по имени;
- передавать в метод сколько угодно значений и не падать, когда их нет.

Дальше — тема 2.5: как метод может изменить переменную снаружи и вернуть сразу несколько значений.